SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code: CSA0604 **Course Title:** Design and Analysis of Algorithms
CO Assessed: CO3 **Assessment:** Assessment Tool 3 – Comparative Analysis
Weightage: 10% of CIA **Total Marks:** 100
Student Name: G Hari Prasad **Reg. No.:** 192525273
Section / Batch: B **Date:** 19-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Bellman-Ford and Floyd-Warshall algorithms for shortest path problems	Q36

SECTION A – Comparative Analysis

Code of Conduct

I G Hari Prasad / 192525273 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G Hari Prasad

RUBRICS FOR CALCULATION

Comparative Analysis

Criteria	Wt.	Level 4	Level 3	Level 2	Level 1
Selection of Studies/Parameters	20%	Selects highly relevant, diverse studies with well-defined parameters	Relevant studies, minor gaps in parameters	Limited or partially relevant selection	Poor or inappropriate selection
Comparison & Differentiation	25%	Clearly compares multiple aspects with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths/weaknesses with strong justification	Evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation, difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4	Level 3	Level 2	Level 1
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date:	Date:	Date:

Q36. Bellman-Ford vs Floyd–Warshall Algorithm (Comparison) — Solution

Name: G Hari Prasad **Reg. No.:** 192525273

Comparison Summary

Aspect	Bellman-Ford	Floyd–Warshall
Approach type	Single-source shortest path	All-pairs shortest path
Method	Edge relaxation, repeated V-1 times	Dynamic programming over intermediate vertices
Time Complexity	O(V·E)	$O(V^3)$
Space Complexity	O(V) — distance array	$O(V^2)$ — full distance matrix
Negative edges	Handles negative weights; detects negative cycles	Handles negative weights; detects negative cycles on the diagonal
Best suited for	Sparse graphs, single source queries	Dense graphs, all-pairs queries

Code Implementation

```
def bellman_ford(graph, V, src):
    dist = [float('inf')] * V
    dist[src] = 0
    for _ in range(V - 1):
        for u, v, w in graph:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    # Check for negative-weight cycles
    for u, v, w in graph:
        if dist[u] != float('inf') and dist[u] + w < dist[v]:
            return None  # negative cycle detected
    return dist

def floyd_warshall(matrix, V):
    dist = [row[:] for row in matrix]
    for k in range(V):
        for i in range(V):
            for j in range(V):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist

if __name__ == "__main__":
    V = 5
    # Edge list format: (u, v, weight)
    edges = [
        (0, 1, 6), (0, 2, 7), (1, 2, 8), (1, 3, 5),
        (1, 4, -4), (2, 3, -3), (2, 4, 9), (3, 1, -2), (4, 3, 7)
    ]

    print('=== Bellman-Ford (source = 0) ===')
    bf_dist = bellman_ford(edges, V, 0)
    for i, d in enumerate(bf_dist):
        print(f'  Distance to {i}: {d}')

    print()
    print('=== Floyd-Warshall (all pairs) ===')
    INF = float('inf')
    matrix = [[0 if i==j else INF for j in range(V)] for i in range(V)]
```

```
    for u, v, w in edges:
        matrix[u][v] = w
    fw_dist = floyd_warshall(matrix, V)
    for i in range(V):
        print(f'  From {i}:', fw_dist[i])
```

Execution Output

```
$ python3 shortest_path_compare.py
=== Bellman-Ford (source = 0) ===
  Distance to 0: 0
  Distance to 1: 2
  Distance to 2: 7
  Distance to 3: 4
  Distance to 4: -2

=== Floyd-Warshall (all pairs) ===
  From 0: [0, 2, 7, 4, -2]
  From 1: [inf, 0, 8, 3, -4]
  From 2: [inf, -5, 0, -3, -9]
  From 3: [inf, -2, 6, 0, -6]
  From 4: [inf, 5, 13, 7, 0]
```

Verification: Bellman-Ford's single-source distances from vertex 0 exactly match row 0 of the Floyd-Warshall all-pairs matrix (0, 2, 7, 4, -2), confirming both algorithms agree on the shortest paths — Bellman-Ford computed only the distances needed from source 0 in $O(V \cdot E)$ time, while Floyd-Warshall computed every pair's distance in $O(V^3)$ time as a byproduct.

Key Takeaways

• Bellman-Ford is the better choice when only shortest paths from a single source are needed, especially on sparse graphs (E close to V), since its $O(V \cdot E)$ cost stays low.
• Floyd-Warshall is preferable when shortest paths between every pair of vertices are needed, since recomputing Bellman-Ford from every vertex would cost $O(V^2 \cdot E)$, which exceeds $O(V^3)$ once the graph is dense.
• Floyd-Warshall's $O(V^2)$ space (a full distance matrix) becomes a real constraint for very large graphs, whereas Bellman-Ford's $O(V)$ space scales far better per query.
• Both algorithms correctly handle negative edge weights and can detect negative-weight cycles, unlike Dijkstra's algorithm, which requires non-negative weights.